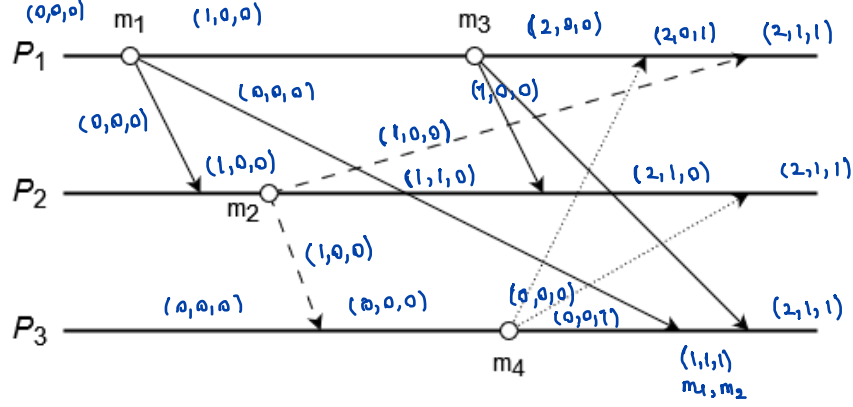


2. (24pts) Consider the execution below in the context of the causal broadcast protocol using vector clocks and answer the following questions. There is no distinction between solid, dashed, and dotted arrows; they are only for visual clarity. Assume messages are delivered as soon as possible, as long as the properties of causal broadcast are not violated. Assume each vector clock is initialized to all zeros.



- (a) (4pts) For each message, give the vector clock attached to that message.
(Notation: $m_1(P_1, P_2)$ means message m_1 from process P_1 to process P_2)
- | | |
|-----------------------------|-----------------------------|
| $m_1(P_1, P_2) : (0, 0, 0)$ | $m_1(P_1, P_3) : (0, 0, 0)$ |
| $m_2(P_2, P_1) : (1, 0, 0)$ | $m_2(P_2, P_3) : (1, 0, 0)$ |
| $m_3(P_1, P_2) : (1, 0, 0)$ | $m_3(P_1, P_3) : (1, 0, 0)$ |
| $m_4(P_3, P_1) : (0, 0, 0)$ | $m_4(P_3, P_2) : (0, 0, 0)$ |
- (b) (12pts) For each pair of messages m and m' , state whether (1) m is causally before m' , (2) m' is causally before m , or (3) m and m' are not causally related. If the messages are causally related, write the property(ies) that relate them.
- | | |
|---|--------------------------------------|
| $m_1, m_2 : (1)m_1 < m_2, \text{InOut}$ | $m_2, m_3 : (3) \text{ not related}$ |
| $m_1, m_3 : (1)m_1 < m_3, \text{FIFO}$ | $m_2, m_4 : (3) \text{ not related}$ |
| $m_1, m_4 : (3) \text{ not related}$ | $m_3, m_4 : (3) \text{ not related}$ |
- (c) (8pts) We wish to measure how often our messages need to buffer across the entire system. For each message m , count the number of times a different message m' is delivered between when m is received and m is delivered. List each m' that is delivered in between.
(e.g. m_7 for P_6 : 2 – P_6 receives m_7 . P_6 delivers m_6 and m_8 . Then, P_6 delivers m_7 .)
- | | |
|---------------------|--|
| m_1 for P_2 : 0 | m_1 for P_3 : 0 |
| m_2 for P_1 : 0 | m_2 for P_3 : 2, $m_4 \rightarrow m_1$ |
| m_3 for P_2 : 0 | m_3 for P_3 : 0 |
| m_4 for P_1 : 0 | m_4 for P_2 : 0 |

4. (20pts) Consider the Consensus Algorithm III we saw in class using the eventual perfect failure detector $\Diamond P$. As a reminder, the leader executes the following in order to decide:

- read the latest adopted value from a majority.
- impose that value at a majority.
- decide and broadcast the decision to all.

In each round diagram,

- draw the messages and the attached value (if any) that are sent in each step,
- clearly write if any process(es) decides during that round,
- the value of each process at the start of the round.

Do this up until round 5, or until every process has decided – whichever happens first, i.e. you do not necessarily need to fill all 5 diagrams.

The following processes crash during the specified rounds. ~~Assume that $\Diamond P$ notifies all processes of the crash at the very start of the round.~~ Also assume that at the start of round i (for $i > 1$), $\Diamond P$ restores any process that crashed in round $(i-1)$; e.g. $\Diamond P$ restores p_1 at the start of round 2. *Leaders are chosen in round robin order.*

- p_1 crashes in Round 1. *after step 1.*
- p_2 crashes in Round 2 *after step 2. p_1 and p_3 crash at the beginning of the round and restore at the end of the round.*
- p_4, p_5 crashes in Round 3. *at the beginning of the round.*
- No crashes in Round 4.
- No crashes in Round 5.

